

Github repo: <https://github.com/RyanDanielLee/FSCT8561-Labs-Ryan-Lee>

Reflection question 1:

A socket is an endpoint that lets a program send and receive data over a network. It acts like a door or connection point between two computers, such as a client and a server.

AF_INET means IPv4 addressing, so the socket uses Internet addresses like 127.0.0.1 and port numbers.

SOCK_STREAM is a reliable, connection-based TCP stream, where data is sent in order like a continuous byte stream.

Part 6 Modifying the message

```
PS C:\Users\rkory\OneDrive\Desktop> python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 63015)
Client says: Hello this is Ryan
PS C:\Users\rkory\OneDrive\Desktop>
```

```
PS C:\Users\rkory\OneDrive\Desktop> python .\client.py
Server replied: Server Received:Hello this is Ryan
PS C:\Users\rkory\OneDrive\Desktop>
```

Server.py

```
import socket

HOST = "127.0.0.1"
PORT = 12345

server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

server_socket.bind((HOST, PORT))
server_socket.listen(1)

print("Server is waiting for a connection...")

client_socket, client_address = server_socket.accept()

print("Connected by:", client_address)

data = client_socket.recv(1024)
```

```
message = data.decode()

print("Client says:", message)
#This is the updated message, you simply just append the message variable
at the end
reply = "Server Received:" + message

client_socket.send(reply.encode())

client_socket.close()
server_socket.close()
```

Client.py

```
import socket

HOST = "127.0.0.1"
PORT = 12345

client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

client_socket.connect((HOST, PORT))
#This is the updated message including my name
message = "Hello this is Ryan"

client_socket.send(message.encode())

response = client_socket.recv(1024)

print("Server replied:", response.decode())

client_socket.close()
```

Part 7

```
PS C:\Users\rkory> cd .\OneDrive\Desktop\  
PS C:\Users\rkory\OneDrive\Desktop> python .\http_client.py  
HTTP/1.1 200 OK  
Date: Thu, 10 Sep 2026 16:20:32 GMT  
Content-Type: text/html  
Transfer-Encoding: chunked  
Connection: close  
Server: cloudflare  
Last-Modified: Wed, 02 Sep 2026 22:14:26 GMT  
Allow: GET, HEAD  
Accept-Ranges: bytes  
Age: 11289  
cf-cache-status: HIT  
CF-RAY: a38fc2b509aa3e7c-YVR  
  
22f  
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-  
scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:  
#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operation  
s.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>  
  
0
```

Reflection Question 2

It connects to “example.com”

Port 80 is used

Port 80 is relevant because it is the standard port used for unencrypted http traffic

The server sends back the http response for the get request with code “200” meaning it was successful. It also sends the response headers and the html content of example.com (This can be seen in the screenshot provided above).

Part 8 Challenge

```
PS C:\Users\rkory\OneDrive\Desktop> python .\server.py  
===== Running on http://127.0.0.1:8080 =====  
(Press CTRL+C to quit)  
Connection established: j8cW6uo3rM_65u4AAAAB  
Socket ID: j8cW6uo3rM_65u4AAAAB  
Data: {'data': 'Hello this is Ryan'}  
Disconnected: j8cW6uo3rM_65u4AAAAB
```

```
PS C:\Users\rkory\OneDrive\Desktop> python .\client.py  
connection established  
Server replied: Server Received:Hello this is Ryan  
disconnected from server
```

Security Reflection

1. Unauthorized users could connect to it and send data/make requests, which would be very bad as it could allow misuse of the server or exposure of sensitive information.
2. No, the lab 0 message is not encrypted, as the application sends the message directly through the connection without adding any encryption mechanism.
3. Yes, because the message is not encrypted, the network traffic could potentially be observed by another system that has the ability to monitor the network through apps like Wireshark.
4. Because client provided data cannot automatically be assumed to be safe. In the textbook, it says that Python does not provide limits or controls for this and that developers are responsible themselves for validating inputs from a security perspective.
5. Because it can allow for the server to determine who is allowed to connect and use its services. Without authentication, an unauthorized client can potentially connect and send messages. Authentication adds an additional layer of protection before allowing access to the server.
6. First way is to use encryption, such as using modern day cryptographic algorithms for encryption.

The second way is to validate client input by checking and restricting the data received from clients before processing it. This will reduce the risk of malicious or unexpected input being processed by the server.

A short paragraph explaining what you learned and what was unclear

I learned how a basic Socket.IO chat uses a client-server model to communicate over a network. I understood that the client connects to the server, sends messages, and receives responses through socket events, which makes communication possible in real time. I also learned that network communication has security considerations, such as protecting messages and controlling who can connect. What was unclear to me was how Socket.IO manages the connection and events behind the scenes, especially how it differs from using a basic Python TCP socket directly.